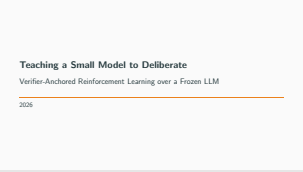
Teaching a Small Model to Deliberate

Verifier-Anchored Reinforcement Learning over a Frozen LLM

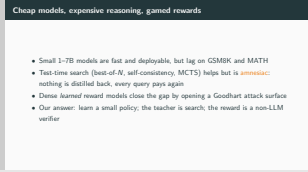
2026



An RL talk: a small agent with an action-value function over a structured state, trained against rewards a deterministic verifier supplies. Everything, MCTS included, is machinery in service of learning a small, fast policy.

Cheap models, expensive reasoning, gamed rewards

- Small 1–7B models are fast and deployable, but lag on GSM8K and MATH
- Test-time search (best-of-N, self-consistency, MCTS) helps but is amnesiac: nothing is distilled back, every query pays again
- Dense *learned* reward models close the gap by opening a Goodhart attack surface
- Our answer: learn a small policy; the teacher is search; the reward is a non-LLM verifier



Inference-time search patches the symptom and never learns from it, and learned rewards get gamed. We turn search into a teacher and anchor reward on a deterministic verifier, paying the search cost once at training, not forever at inference.

Thesis

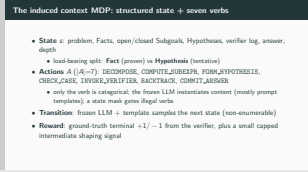
1. Freeze the base LLM; learn small heads π_θ, Q_ψ over a structured state
2. Cast LLM deliberation on verifiable tasks as approximate Q-learning over an induced context MDP
3. Reward is verifier-anchored, not a learned critic
4. MCTS is a sample-efficient *policy-improvement operator* supplying stronger targets, not the central algorithm

Thesis
1. Freeze the base LLM; learn small heads π_θ, Q_ψ over a structured state
2. Cast LLM deliberation on verifiable tasks as approximate Q-learning over an induced context MDP
3. Reward is verifier-anchored, not a learned critic
4. MCTS is a sample-efficient *policy-improvement operator* supplying stronger targets, not the central algorithm

The whole talk in one slide. The learnable object is the agent and its action-value function. Search is subordinate, producing better samples and targets. Because the LLM is frozen, any gain is attributable to the heads and the search.

The induced context MDP: structured state + seven verbs

- **State** s: problem, Facts, open/closed Subgoals, Hypotheses, verifier log, answer, depth
  - load-bearing split: **Fact** (proven) vs **Hypothesis** (tentative)
- **Actions** A ($|A|$=7): `DECOMPOSE`, `COMPUTE_SUBEXPR`, `FORM_HYPOTHESIS`, `CHECK_CASE`, `INVOKE_VERIFIER`, `BACKTRACK`, `COMMIT_ANSWER`
  - only the verb is categorical; the frozen LLM instantiates content (mostly prompt templates); a state mask gates illegal verbs
- **Transition**: frozen LLM + template *samples* the next state (non-enumerable)
- **Reward**: ground-truth terminal $+1/-1$ from the verifier, plus a small capped intermediate shaping signal



The state schema is the Markov bet; Fact-vs-Hypothesis lets the agent hold tentative claims apart from proven ones. The verb stays categorical so the Q-head has a fixed output and search a stable branching factor. The transition is sampled-only, which later forces sampled chance nodes.

A concrete trace (GSM8K)

P: "A store sold 7 boxes with 6 pencils each. How many pencils?"

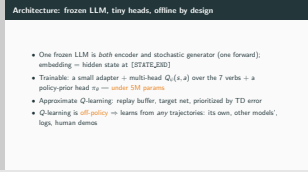
1. `FORM_HYPOTHESIS`: `hyp_1` = "the total is 42" (tentative)
2. `COMPUTE_SUBEXPR` "7×6" $\rightarrow$ 42 (derived Fact)
3. `INVOKE_VERIFIER` on `hyp_1` (SymPy) $\rightarrow$ confirmed; promoted to Fact ($+0.05$, capped)
4. `COMMIT_ANSWER` "42" $\rightarrow$ terminal verifier $\rightarrow$ reward $+1$

A concrete trace (GSM8K)
P: "A store sold 7 boxes with 6 pencils each. How many pencils?"
1. FORM_HYPOTHESIS: hyp_1 = "the total is 42" (tentative)
2. COMPUTE_SUBEXPR "7 × 6" → 42 (derived Fact)
3. INVOKE_VERIFIER on hyp_1 (SymPy) → confirmed; promoted to Fact (+0.05, capped)
4. COMMIT_ANSWER "42" → terminal verifier → reward +1

State, verb-actions, sampled transitions, and verifier reward, all concrete. The small promotion reward is hard-capped so it cannot be farmed above the terminal. At inference $B{=}0$ walks this greedily; $B{>}0$ runs MCTS.

Architecture: frozen LLM, tiny heads, offline by design

- One frozen LLM is *both* encoder and stochastic generator (one forward); embedding = hidden state at `[STATE_END]`
- Trainable: a small adapter + multi-head $Q_\psi(s, a)$ over the 7 verbs + a policy-prior head π_θ — under 5M params
- Approximate Q-learning: replay buffer, target net, prioritized by TD error
- Q-learning is off-policy $\Rightarrow$ learns from *any* trajectories: its own, other models', logs, human demos



A single forward gives both the embedding and the verb-instantiator continuation. We train only adapter and heads, so representation drift is bounded and gains are attributable to the heads and search. Off-policy is the quiet superpower: data source and learner are decoupled.

MCTS as a policy-improvement operator, and the loop

PUCT (learned policy as prior, Q_ψ as leaf value):

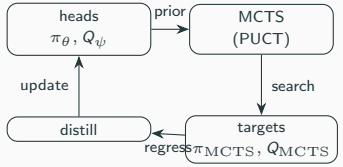
$$\arg\max_a \ \widehat{Q}(s,a) + c_{\text{puct}}\,\pi_\theta(a \mid s)\frac{\sqrt{N(s)}}{1+N(s,a)}$$

Chance-node mean is an honest Bellman estimator:

$$\frac{W(s,a)}{N(s,a)} \approx \sum_{s'} P(s' \mid s,a)[R+\gamma V(s')]$$

Distill targets: π_{MCTS} (visits), Q_{MCTS} (chance-Q);

$$\mathcal{L}_{\text{total}} = \left(Q_\psi - Q_{\text{MCTS}}\right)^2 + \lambda\,\mathcal{L}_\pi$$



System 1 heads, System 2 search; training shifts competence into System 1.

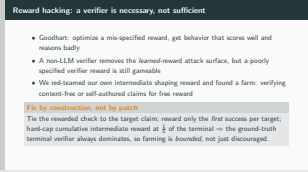
MCTS replaces the intractable Bellman sum with sampled rollouts at a query state, the sampled chance node being the only available estimator. The same Q-learning update runs; only the targets get better because they average over search-guided trajectories. AlphaZero's structure inverted: the agent and Q are the center, search is subordinate.

Reward hacking: a verifier is necessary, not sufficient

- Goodhart: optimize a mis-specified reward, get behavior that scores well and reasons badly
- A non-LLM verifier removes the *learned*-reward attack surface, but a poorly specified verifier reward is still gameable
- We red-teamed our own intermediate shaping reward and found a farm: verifying content-free or self-authored claims for free reward

Fix by construction, not by patch

Tie the rewarded check to the target claim; reward only the *first* success per target; hard-cap cumulative intermediate reward at $\frac{1}{5}$ of the terminal $\Rightarrow$ the ground-truth terminal verifier always dominates, so farming is *bounded*, not just discouraged.



A verifiable reward is necessary but not sufficient: a badly specified shaping term is still hackable. We caught exactly this via adversarial review. The fix is structural, capping the cumulative shaping below the terminal so no farm can rival actually solving. Red-teaming the reward is part of the method.

Inference and deployment: one compute knob, always-on learning

$B = 0$ **(fast)**	$B > 0$ **(search)**
masked arg max over Q_ψ; one forward/decision; constant latency	PUCT MCTS for B sims; sample the visit-count policy

- Same trained heads; one dial traces a compute-vs-accuracy frontier, no retraining
- Off-policy + verifier-anchored $\Rightarrow$ keep learning in deployment from any verifier-style signal, incl. user thumbs up/down
- Per-user adapters absorb idiosyncratic feedback; the verifier stays the anchor that bounds drift

Two modes, one learned actor with different search on top; B becomes a user-controlled dial. Because the heads were distilled from MCTS targets, fast mode already carries much of the benefit. Deployment traffic is just more off-policy experience, so the model can keep improving, anchored by the verifier.

Positioning and defensible novelty

- **AlphaZero / expert iteration**: we invert it — agent and Q_ψ central, search optional
- **Verifier-anchored reasoning** (PRMs, AlphaProof, DeepSeek-R1): verifier-vs-aesthetic asymmetry is load-bearing; `INVOKE_VERIFIER` is a callable verb
- **VerifierQ**: verifier-as-*action* not verifier-as-Q; online distilled targets; categorical multi-head Q
- **vs STaR / ReST-EM / Math-Shepherd**: per-step Q_{MCTS} as a genuine Bellman target, not trajectory-level or a process-reward model

Positioning and defensible novelty
- AlphaZero / expert iteration: we invert it — agent and Q_ψ central, search optional
- Verifier-anchored reasoning (PRMs, AlphaProof, DeepSeek-R1): verifier-vs-aesthetic asymmetry is load-bearing; INVOKE_VERIFIER is a callable verb
- VerifierQ: verifier-as-action not verifier-as-Q; online distilled targets; categorical multi-head Q
- vs STaR / ReST-EM / Math-Shepherd: per-step Q_{MCTS} as a genuine Bellman target, not trajectory-level or a process-reward model

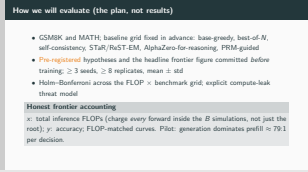
We inherit AlphaZero's joint search-and-learning but invert its center, so non-search baselines are first-class. The afterstate split is borrowed, not claimed novel; novelty is its application to the non-enumerable LLM-induced transition.

How we will evaluate (the plan, not results)

- GSM8K and MATH; baseline grid fixed in advance: base-greedy, best-of-N, self-consistency, STaR/ReST-EM, AlphaZero-for-reasoning, PRM-guided
- Pre-registered hypotheses and the headline frontier figure committed *before* training; ≥ 3 seeds, ≥ 8 replicates, mean $\pm$ std
- Holm–Bonferroni across the FLOP $\times$ benchmark grid; explicit compute-leak threat model

Honest frontier accounting

x: total inference FLOPs (charge *every* forward inside the B simulations, not just the root); y: accuracy; FLOP-matched curves. Pilot: generation dominates prefill $\approx$ 79:1 per decision.



Methodology, not a victory lap; we deliberately do not lean on current pilot numbers, and strong results are in progress. Honest FLOP accounting is what makes beating best-of-N mean we learned something, not that we spent more compute.

Vision

A small, fast model that learns to deliberate over structured states,
anchored by verifiable (non-learned) rewards.

Offline today, online-continual in deployment, one test-time compute knob.

MCTS earns its keep only as the operator that manufactures stronger targets.

Vision
A small, fast model that learns to deliberate over structured states,
anchored by verifiable (non-learned) rewards.
Offline today, online-continual in deployment, one test-time compute knob.
MCTS earns its keep only as the operator that manufactures stronger targets.

The takeaway is the RL-first frame: learn a small policy and Q over a structured state with verifiable rewards; search teaches the heads. Fast by default, thinks harder on demand, keeps learning after deployment. Next: the pre-registered frontier at scale.